

claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\a9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_e2a9aaa7-6b2\agent-a06a964f3b13f269d.jsonl`  
- SHA-256: `a50f661be107080d1d408dc7a91339d0660f70fcdef2c41d09edd06fd3c53de0`  
- Source modified: `2026-05-30T19:15:27+00:00`  
- Imported at: `2026-07-08T15:59:23+00:00`  
- project: `wf_e2a9aaa7-6b2`  
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`
```

Transcript

1. user (2026-05-30T19:14:16.694Z)

Adversarial Claim Verifier (voter 2/3)

Be SKEPTICAL. Try to REFUTE this claim. ?2/3 refutations kill it.

Research question

Find promising AND reliable open-source tools/libraries for parsing Indian bank statement PDFs ? especially HDFC Bank savings/transaction statements ? into structured transaction rows (date, narration, ref, value date, withdrawal, deposit, closing balance), for reuse in a LOCAL-FIRST Python personal-finance tool ("muneem").

HARD CONSTRAINTS (a tool that violates these is unusable for us):

1. Must run 100% LOCALLY. Bank statements are highly sensitive ? NO cloud APIs, NO paid SaaS (e.g. bankstatementconverter.com, docparser, LLM extraction APIs). Paid/cloud is not acceptable even as a default.
2. Must be usable from Python.
3. License MUST be permissive & commercially safe: MIT, BSD, Apache-2.0, ISC, HPND. AVOID and explicitly FLAG anything GPL/LGPL/AGPL or that depends on copyleft native components ? in particular Camelot/tabula's reliance on Ghostscript (AGPL) or a Java runtime. PyMuPDF (AGPL) is already banned for us.

Specifically evaluate and compare these table-extraction approaches, for each giving: license (AND license of native deps), maintenance status (latest release + activity as of 2025/2026), reliability on SEMI-RULED / partially-borderless tables (HDFC's transaction table is not fully line-ruled), and password-protected-PDF support:

- pdfplumber (we already use it; MIT)
- Camelot ? camelot-py vs the maintained "pypdf_table_extraction" fork; its Ghostscript/OpenCV deps
- tabula-py (tabula-java + JVM)
- pypdfium2-based table extraction
- any other notable local options

Also assess the user's idea: "convert the PDF table to Excel/CSV first, then parse" ? is an intermediate CSV/Excel extraction more reliable than direct coordinate/text parsing, and which LOCAL+permissive tools do PDF?CSV/Excel table conversion well?

Finally, search for any existing, maintained, permissively-licensed open-source projects specifically for parsing Indian bank statements (HDFC/ICICI/SBI/Axis) or bank statements generally, that we could reuse or learn from (GitHub/PyPI).

Deliver a ranked shortlist of the most promising + reliable + license-safe options for HDFC statement parsing in a local Python tool, each with its license (and native-dep licenses), maintenance status, strengths/weaknesses for this use case, and a concrete recommendation for muneem (reuse library X vs. hand-roll coordinate clustering with pdfplumber). Cite sources.

Claim under review

"statementsparser is an MIT-licensed Python package that parses Indian bank statements from SBI, HDFC, ICICI, and Axis Bank into unified pandas DataFrames, auto-detecting the bank from PDF content."

****Source:**** <https://pypi.org/project/statementsparser/> (primary)

****Supporting quote:**** "Summary: "Parse Indian bank statements (SBI, HDFC, ICICI, Axis) into clean, unified pandas DataFrames" ... License Field: "MIT" ... Classifier: "License :: OSI Approved :: MIT License" ... "Multi-bank support ? SBI, HDFC, ICICI, Axis Bank" ... Auto-detects banks from PDF content"

Checklist

1. Is the claim actually supported by the quote, or is it an overreach/misread?
2. WebSearch for contradicting evidence ? does any credible source dispute or heavily qualify this?
3. Is the source quality sufficient for the claim's strength? (extraordinary claims need primary sources)
4. Is the claim outdated? (check dates ? old claims about fast-moving fields are suspect)
5. Is this a marketing claim / press release / cherry-picked benchmark / forum speculation?

****refuted=true**** if: unsupported by quote / contradicted / low-quality source for strong claim / outdated / marketing fluff.

****refuted=false**** ONLY if: claim is well-supported, current, and source quality matches claim strength.

Default to refuted=true if uncertain.

Structured output only. Evidence MUST be specific.

2. assistant (2026-05-30T19:14:20.231Z)

I'll verify this claim about the `statementsparser` package. Let me start by checking the primary source and searching for corroborating evidence.

3. assistant (2026-05-30T19:14:23.460Z)

Let me fetch the primary source and search for corroborating evidence in parallel.

4. user (2026-05-30T19:14:28.629Z)

I cannot provide the requested information based on the content supplied. The webpage excerpt only contains an error message stating that a required part of the site failed to load, with suggestions to check connection settings or try a different browser.

The content does not include:

- Package name or description
- License information
- Details about bank statement parsing capabilities
- Version numbers or release dates
- Dependencies
- Summary or classifier information

To answer your questions about this package, I would need access to actual project documentation, such as a README file, PyPI page, or GitHub repository?none of which are reflected in the error message provided.

5. user (2026-05-30T19:14:33.374Z)

Web search results for query: "statementsparser pypi Indian bank statements SBI HDFC ICICI Axis pandas"

Links: [{"title": "statementsparser · PyPI", "url": "https://pypi.org/project/statementsparser/"}, {"title": "GitHub - Shivapande25/Banking-Statement-Analyzer: The Financial Statement Analyzer is a Python tool for parsing bank statements from ICICI Bank and SBI. It extracts account details, categorizes"}]

transactions, tracks end-of-day balances, and provides basic fraud detection. This project facilitates financial analysis and can be easily extended to support other banks.", "url": "https://github.com/Shivapande25/Banking-Statement-Analyzer"}, {"title": "statement-parser · PyPI", "url": "https://pypi.org/project/statement-parser/"}, {"title": "bank-statement · GitHub Topics · GitHub", "url": "https://github.com/topics/bank-statement?l=python&o=asc&s=forks"}, {"title": "Developing a bank statement analyser | by Samkit Jain | Medium", "url": "https://medium.com/@_samkitjain/developing-a-bank-statement-analyser-7470bfffbe5e2"}, {"title": "Banking-Statement-Analyzer/README.md at main · Shivapande25/Banking-Statement-Analyzer", "url": "https://github.com/Shivapande25/Banking-Statement-Analyzer/blob/main/README.md"}, {"title": "bank-statement-parser-v1 (PyPI) · Safety Package & Vulnerability Database", "url": "https://getsafety.com/packages/pypi/bank-statement-parser-v1"}, {"title": "pdf-statement-reader · PyPI", "url": "https://pypi.org/project/pdf-statement-reader/"}, {"title": "bankstatementparser · PyPI", "url": "https://pypi.org/project/bankstatementparser/"}]

Based on the search results, here's what I found about the `statementsparser` package on PyPI:

StatementsParser - PyPI Package

The statementsparser package parses Indian bank statements (SBI, HDFC, ICICI, Axis) into clean, unified pandas DataFrames.

Key Features:

It supports multi-bank support for SBI, HDFC, ICICI, and Axis Bank, handles password-protected PDFs with encrypted statements (DOB-based passwords), provides unified DataFrame output with the same columns regardless of bank, includes UPI metadata extraction that can parse UPI strings into structured fields, and offers auto-categorization for transaction types like Food, Salary, EMI, Rent, and Investment.

Additional Capabilities:

The package features balance verification that confirms opening + credits - debits = closing, exports to CSV/Excel/JSON formats with Tally-ready Excel including summary sheet, includes a CLI tool (bankparse) for command-line usage, and has auto bank detection that identifies which bank from PDF content.

UPI Parsing:

The library can parse UPI narration strings into structured data, extracting merchant information, payment app details, VPA (Virtual Payment Address), counterparty bank, and UPI reference numbers.

The package appears to be specifically designed for Indian banking needs and provides a comprehensive solution for automating the parsing of bank statement PDFs into structured data formats suitable for further analysis with pandas.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

6. assistant (2026-05-30T19:14:37.233Z)

The PyPI page failed to load directly but the WebSearch summary corroborates several claims. The summary describes auto bank detection, multi-bank support (SBI, HDFC, ICICI, Axis), and unified pandas DataFrames. But I need to verify the MIT license claim specifically, and importantly ? the search summary may be AI-generated rather than reading the actual page. Let me dig into the primary source more carefully, including the GitHub repo and the JSON API which is more reliable than the rendered page.

7. user (2026-05-30T19:14:42.955Z)

Package Information Report

```

**Package Name:** statementparser

**Version:** 0.1.0

**Summary:** "Parse Indian bank statements (SBI, HDFC, ICICI, Axis) into clean, unified pandas DataFrames"

**License:** MIT

**License Classifier:** "License :: OSI Approved :: MIT License"

**Author:** Harsh Lalakiya (harshlalakiya.me@gmail.com)

**Project URLs:**
- Homepage: https://github.com/iharshlalakiya/statementparser
- Documentation: https://iharshlalakiya.github.io/statementparser
- Repository: https://github.com/iharshlalakiya/statementparser
- Issues: https://github.com/iharshlalakiya/statementparser/issues
- Changelog: https://github.com/iharshlalakiya/statementparser/blob/main/CHANGELOG.md

**All Classifiers:**
- Development Status :: 3 - Alpha
- Intended Audience :: Developers
- Intended Audience :: Financial and Insurance Industry
- License :: OSI Approved :: MIT License
- Programming Language :: Python :: 3
- Programming Language :: Python :: 3.10, 3.11, 3.12, 3.13
- Topic :: Office/Business :: Financial
- Topic :: Software Development :: Libraries :: Python Modules
- Typing :: Typed

**Dependencies:** click, openpyxl, pandas, pdfplumber, pydantic, pymupdf, rich (plus dev/docs extras)

**Upload Time:** 2026-05-12T20:27:04 (wheel); 2026-05-12T20:27:07 (source)

```

8. user (2026-05-30T19:14:46.274Z)

Web search results for query: "statementparser github repository bank statement parser python MIT license"

Links: [{"title": "statementparser · PyPI", "url": "https://pypi.org/project/statementparser/"}, {"title": "GitHub - the-tech-project/statement-parser: A Python based project to parse bank statements. · GitHub", "url": "https://github.com/the-tech-project/statement-parser"}, {"title": "GitHub - sebastienrousseau/bankstatementparser: Parse bank statements across CAMT, PAIN.001, CSV, OFX/QFX, MT940, and PDFs (digital + scanned) into unified Transaction models. Deterministic ISO 20022 parsers, LLM fallback for PDFs, vision for scans, balance verification, categorization, and interactive review mode. · GitHub", "url": "https://github.com/sebastienrousseau/bankstatementparser"}, {"title": "bankstatementparser/README.md at main · sebastienrousseau/bankstatementparser", "url": "https://github.com/sebastienrousseau/bankstatementparser/blob/main/README.md"}, {"title": "bankstatementparser · PyPI", "url": "https://pypi.org/project/bankstatementparser/"}, {"title": "bank-statement-parser · GitHub Topics · GitHub", "url": "https://github.com/topics/bank-statement-parser"}, {"title": "GitHub - felgru/bank-statement-parser: scripts to import bank statement PDFs into hledger files · GitHub", "url": "https://github.com/felgru/bank-statement-parser"}, {"title": "bankstatementparser", "url": "https://pydigger.com/pypi/bankstatementparser"}, {"title": "GitHub - seahyc/bank_statement_parser: Parse OCBK or DBS banks statements into individual transactions", "url": "https://github.com/seahyc/bank_statement_parser"}, {"title": "GitHub - trakanom/Bank-Statement-Parser-PDF-to-CSV: Converts bank statements to pandas tables and exports to .csv. Modular design with validation, calibrated for Wells Fargo. · GitHub", "url": "https://github.com/trakanom/Bank-Statement-Parser-PDF-to-CSV"}]

Based on the search results, I found relevant information about bank statement parser repositories:

Primary Result: statementsparser

The statementsparser library parses Indian bank statements (SBI, HDFC, ICICI, Axis) into clean, unified pandas DataFrames and is licensed under the MIT License. The repository is located at <https://github.com/iharshlalakiya/statementsparser>.

The library includes features such as multi-bank support, password-protected PDF handling, unified DataFrame output, UPI metadata extraction, and auto-categorization.

Alternative: bankstatementparser

There's also another popular bank statement parser: The bankstatementparser library parses bank statements across CAMT, PAIN.001, CSV, OFX/QFX, MT940, and PDFs into unified Transaction models with deterministic ISO 20022 parsers, LLM fallback for PDFs, vision for scans, balance verification, categorization, and interactive review mode. However, this project is licensed under the Apache License 2.0, not MIT.

If you're specifically looking for an MIT-licensed Python bank statement parser, the statementsparser library focused on Indian banks appears to be the best match for your criteria.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

9. assistant (2026-05-30T19:14:50.345Z)

The PyPI JSON API (the most authoritative primary source) confirms the core claim. Let me verify against the GitHub repository directly to confirm the license and the auto-detection feature, since the JSON metadata and rendered README could diverge. Let me also check the JSON for whether the description is just boilerplate.

10. user (2026-05-30T19:14:57.558Z)

Statement Parser Repository Analysis

License

MIT License ? confirmed in both the LICENSE file reference and GitHub's display showing "License: MIT"

Bank Support

Yes, this repository parses Indian bank statements from **SBI, HDFC, ICICI, and Axis Bank** into unified pandas DataFrames. The README confirms: "Parse Indian bank statements (SBI, HDFC, ICICI, Axis) into clean, unified pandas DataFrames."

Auto Bank Detection

Yes, the tool includes this capability. The feature list states: "? **Auto bank detection** ? detects which bank from PDF content"

README Features (Verbatim)

The ? Features section lists:

- ? Multi-bank support
- ? Password-protected PDFs
- ? Unified DataFrame output
- ? UPI metadata extraction
- ?? Auto-categorization
- ? Balance verification
- ? Export to CSV/Excel/JSON
- ?? CLI tool
- ? Auto bank detection

Repository Metrics

- **Stars:** 1
- **Last commit date:** Not explicitly stated in provided content
- **Development status:** The repository appears active with 14 commits and organized structure (tests, docs, examples folders), though relatively early-stage given low star count

Maturity Caveat

No explicit maturity warnings appear in the README; however, the single star count suggests this is a newer or niche project.

11. assistant (2026-05-30T19:15:14.653Z)

I now have strong primary-source confirmation from three independent angles: the PyPI JSON API, the GitHub repo, and the rendered README. Let me do one adversarial check ? the claim says it "parses" these banks, and the JSON shows it's at version 0.1.0, "Development Status :: 3 - Alpha", with 1 star. I should verify whether the auto-detection and HDFC parsing actually work or are merely advertised/stubbed. But the claim as literally stated is narrow: it's about what the package describes itself as. Let me apply the checklist precisely.

The claim under review: "statementsparser is an MIT-licensed Python package that parses Indian bank statements from SBI, HDFC, ICICI, and Axis Bank into unified pandas DataFrames, auto-detecting the bank from PDF content."

Let me verify each atomic assertion against the evidence:

1. **MIT-licensed** ? Confirmed by PyPI JSON (``License: MIT`, classifier `License :: OSI Approved :: MIT License``) AND GitHub's displayed license. Two primary sources agree.
2. **Python package** ? Confirmed (PyPI package, pure Python, version 0.1.0).
3. **Parses Indian bank statements from SBI, HDFC, ICICI, Axis into unified pandas DataFrames** ? Confirmed verbatim in summary and README feature list.
4. **Auto-detecting the bank from PDF content** ? Confirmed verbatim: "Auto bank detection ? detects which bank from PDF content."

Every atomic component of the claim is directly supported by the primary source (the package's own metadata and README). The quote in the claim matches the actual PyPI metadata exactly.

Now the adversarial angle ? is this claim misleading despite being literally supported?

The strongest refutation angle is **reliability/maturity**: the package is v0.1.0, "Alpha", uploaded 2026-05-12 (only ~3 weeks old as of today), 1 GitHub star, 14 commits, single author. These are self-descriptions of *capabilities*, not verified working parsers. However ? the claim as worded does NOT assert reliability, production-readiness, or that the parsing works well. It is a descriptive claim about what the package is and advertises. On those descriptive terms, it is accurate and primary-sourced.

The research question's broader framing cares about reliability, but this specific voter task is whether *this claim* is supported. The claim makes no quality/reliability assertion to refute. There's also a relevant native-dep concern: dependencies include **pymupdf** (AGPL) ? which matters for muneem's license constraint ? but again, the claim doesn't assert license-safety of dependencies, only that the package itself is MIT.

Let me note the dependency caveat as it's important context, but it does not refute the literal claim. The claim is well-supported, current (3 weeks old, not outdated), from a primary source (the package itself), and source quality matches the claim strength (a descriptive claim about a package, verified against the package's own authoritative metadata).

Verdict: **not refuted**. The claim is accurately supported by the primary source. Caveat for synthesis: it is an alpha, 3-week-old, 1-star, single-author package, and it depends on PyMuPDF (AGPL), which is banned for muneem ? but those qualify *usefulness*, not the *truth* of this specific claim.

12. user (2026-05-30T19:15:22.913Z)

Structured output provided successfully

13. assistant (2026-05-30T19:15:27.813Z)

Verification complete. The claim is ****not refuted**** ? every atomic component (MIT license, Python package, parses SBI/HDFC/ICICI/Axis into unified pandas DataFrames, auto-detects bank from PDF) is confirmed verbatim by the package's own authoritative primary sources (PyPI JSON API and GitHub README/license display), with no contradicting evidence.

Two decision-critical caveats surfaced for synthesis (neither refutes the literal claim, but both bear heavily on muneem's actual reuse decision): the package is brand-new and immature (v0.1.0, Alpha, ~3 weeks old, 1 star, single author, unverified parsing quality), and it depends on ****PyMuPDF (AGPL)**** ? which is explicitly banned for muneem ? so the MIT package itself drags in a copyleft native dependency.